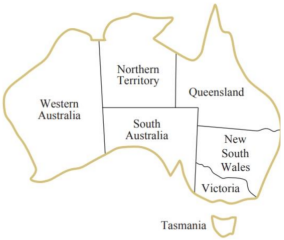
Inteligență Artificială

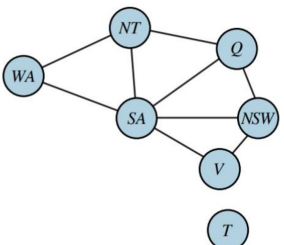
Problema satisfacerii restricțiilor

Slides: Andrei Olaru, Adina Florea

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

- problema colorării hărților





$WA, NT, Q, SA, NSW, V, T \in Colors$
$WA \neq NT, WA \neq SA, NT \neq SA, \ldots$

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

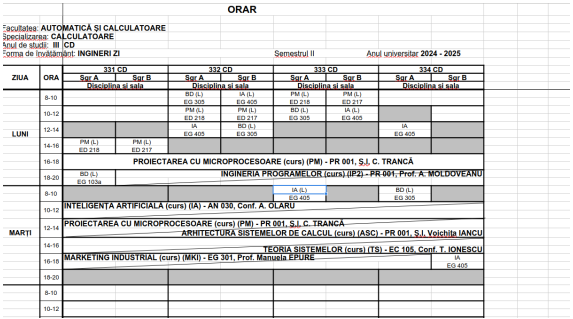
- problema colorării hărților
- sudoku

4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i

$a, b, \ldots, i \in \{1, 2, 3, 4\}$
$a \notin \{1, 2, 4, d, i\}$
$b \notin \{2, 4, c, e, h\}$
$c \notin \{2, 4, b, c, f\}$
$d \notin \{1, 3, 4, a, g\}$
$\ldots$

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

- problema colorării hărților
- sudoku
- orarul



ORAR

Facultatea: AUTOMATICĂ ȘI CALCULATOARE
Specializarea: CALCULATOARE
Anul de studii: III CD
Forma de învățământ: INGINERI ZI
Semestrul II
Anul universitar 2024 - 2025

ZIUA	ORA	331 CD		332 CD		333 CD		334 CD	
		Sgr A	Sgr B	Sgr A	Sgr B	Sgr A	Sgr B	Sgr A	Sgr B
		Disciplina si sala		Disciplina si sala		Disciplina si sala		Disciplina si sala	
LUNI	8-10			BD (L) EG 303	IA (L) EG 405	PM (L) ED 218	PM (L) ED 217		
	10-12			PM (L) ED 218	PM (L) ED 217	BD (L) EG 305	IA (L) EG 405		
	12-14			IA EG 405	BD (L) EG 305			IA EG 405	
	14-16	PM (L) ED 218	PM (L) ED 217						
	16-18	PROIECTAREA CU MICROPROCESOARE (curs) (PM) - PR 001, S.I. C. TRANCĂ							
	18-20	BD (L) EG 103a	INGINERIA PROGRAMELOR (curs) (IP2) - PR 001, Prof. A. MOLDOVEANU						
MARȚI	8-10					IA (L) EG 405		BD (L) EG 305	
	10-12	INTELIGENȚA ARTIFICIALĂ (curs) (IA) - AN 030, Conf. A. OLARU							
	12-14	PROIECTAREA CU MICROPROCESOARE (curs) (PM) - PR 001, S.I. C. TRANCĂ							
	14-16	ARHITECTURA SISTEMELOR DE CALCUL (curs) (ASC) - PR 001, S.I. Voichița IANCU							
	16-18	TEORIA SISTEMELOR (curs) (TS) - EC 105, Conf. T. IONESCU							
	18-20	MARKETING INDUSTRIAL (curs) (MKI) - EG 301, Prof. Manuela EPURE						IA EG 405	
	8-10								
	10-12								

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

- problema colorării hărților
- sudoku
- orarul
- problema lui Einstein (Zebra puzzle)



	House 1	House 2	House 3	House 4	House 5
Colour					
Nationality					
Drink					
Sport					
Pet					

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

- problema colorării hărților
- sudoku
- orarul
- problema lui Einstein (Zebra puzzle)
- puzzle-uri matematice

$$\begin{array}{cccc} & T & W & O \\ + & T & W & O \\ \hline F & O & U & R \end{array}$$

$$W, O, U, R \in 0 \ldots 9$$

$$T, F \in 1 \ldots 9$$

$$2 \cdot O \,\%\, 10 = R$$

$$2 \cdot W \,\%\, 10 + 2 \cdot O \div 10 = U$$

$$2 \cdot T \,\%\, 10 + 2 \cdot W \div 10 = O$$

$$2 \cdot T \div 10 = F$$

Problema satisfacerii restricțiilor
(Constraint Satisfaction Problem – CSP)

- problema colorării hărților
- sudoku
- orarul
- problema lui Einstein (Zebra puzzle)
- puzzle-uri matematice
- ...

$\{X_1 \dots X_N\} = V$ – *Variable*

$\{D_1 \dots D_N\}$ – *Domenii*

$\{R_1 \dots R_k\} = R$ – *Restricții*, $R_j = \langle t_j, r_j \rangle, t_j \subseteq V, r_j : \times_{X_i \in t_j} D_i \to \{True, False\}$

$\{\langle X_1, x_1 \rangle \dots \langle X_N, x_N \rangle\}$ – Atribuire la *valori*, cu $x_i \in D_i \cup \{\perp\}$ ($\perp \equiv$ neatribuit la o valoare)

o atribuire este *soluție* dacă

- $x_i \in D_i, i = \overline{1, N}$, i.e. toate variabilele au atribuită o valoare (*atribuire completă*)
- $\forall R_j \in R, r_j(vals) = True$, cu *vals* valorile variabilelor din t_j (*soluție validă*)

- CSP totală – soluția este completă și nu încalcă nicio restricție

- CSP totală – soluția este completă și nu încalcă nicio restricție
- CSP parțială – soluția este completă dar încalcă unele restricții

  $R_j = \langle t_j, r_j, c_j \rangle$, cu c_j costul restricției R_j

  costul soluției $c = \sum_{R_j \in R, r_j(vals(t_j))=False} c_j$

- CSP totală – soluția este completă și nu încalcă nicio restricție
- CSP parțială – soluția este completă dar încalcă unele restricții

  $R_j = \langle t_j, r_j, c_j \rangle$, cu c_j costul restricției R_j

  costul soluției $c = \sum_{R_j \in R, r_j(vals(t_j))=False} c_j$

- CSP binară – toate restricțiile sunt binare

  $\forall R_j \in R, t_j \in V \times V$

Backtracking

- CSP este o problemă NP, apropiată de problema SAT

$\Rightarrow$ CSP trebuie rezolvat prin backtracking

- ce putem face este să încercăm să *reducem* spațiul de căutare și factorul de ramificare
  - îmbunătățirea consistenței căutării
  - reducerea spațiului de căutare

CSP-BKT

Intrări: *Vars* – variabile rămase de atribuit, inițial X;
Sol – soluția (atribuirea) parțială, inițial $\emptyset$

Ieșire: O atribuire completă sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci întoarce** *Sol*
2. $X_i \leftarrow$ *alege_variabila*(*Vars*)
3. **pentru fiecare** $x \in D_i$
4. **dacă** *consistent*$(Sol \cup \langle X_i, x \rangle, R)$ **atunci**
5. $res \leftarrow$ *CSP-BKT*$(Vars \setminus \{X_i\}, Sol \cup \langle X_i, x \rangle)$
6. **dacă** $res \neq$ EȘEC **atunci întoarce** *res*
7. **întoarce** EȘEC

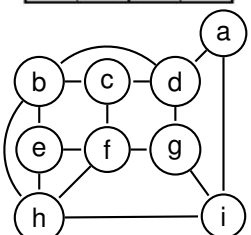
Propagarea locală a restricțiilor

Intuitiv:

- inițial, după aplicarea restricțiilor unare, avem
  - $a \in \{3\}$
  - $d \in \{2\}$
  - $b, c, e, f \in \{1, 3\}$
  - $h \in \{1, 2\}$
  - $g \in \{4\}$
  - $i \in \{1\}$

1. a poate fi doar 3; a se învecinează cu d, i; ambele au domenii compatibile cu valoarea lui a

4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i

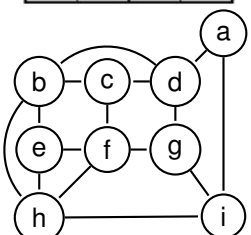


Intuitiv:

- inițial, după aplicarea restricțiilor unare, avem
  - $a \in \{3\}$
  - $d \in \{2\}$
  - $b, c, e, f \in \{1, 3\}$
  - $h \in \{1, 2\}$
  - $g \in \{4\}$
  - $i \in \{1\}$

1. a poate fi doar 3; a se învecinează cu d, i; ambele au domenii compatibile cu valoarea lui a
2. d poate fi doar 2, se învecinează cu b, c, g; sunt compatibile

4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i

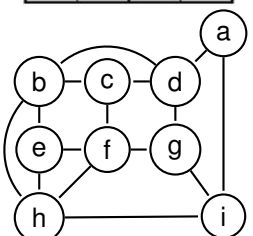


Intuitiv:

- inițial, după aplicarea restricțiilor unare, avem
  - $a \in \{3\}$
  - $d \in \{2\}$
  - $b, c, e, f \in \{1, 3\}$
  - $h \in \{1, 2\}$
  - $g \in \{4\}$
  - $i \in \{1\}$

1. a poate fi doar 3; a se învecinează cu d, i; ambele au domenii compatibile cu valoarea lui a
2. d poate fi doar 2, se învecinează cu b, c, g; sunt compatibile
3. i poate fi doar 1, se învecinează cu h, deci h nu poate fi 1 $\Rightarrow$ restricționăm domeniul lui h la $\{2\}$

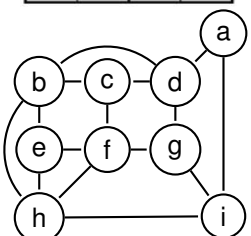
4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i



Intuitiv:

- inițial, după aplicarea restricțiilor unare, avem
  - $a \in \{3\}$
  - $d \in \{2\}$
  - $b, c, e, f \in \{1, 3\}$
  - $h \in \{1, 2\}$
  - $g \in \{4\}$
  - $i \in \{1\}$

4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i

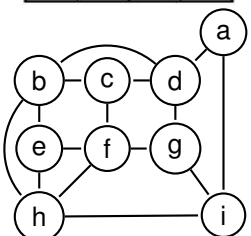


1. a poate fi doar 3; a se învecinează cu d, i; ambele au domenii compatibile cu valoarea lui a
2. d poate fi doar 2, se învecinează cu b, c, g; sunt compatibile
3. i poate fi doar 1, se învecinează cu h, deci h nu poate fi 1 $\Rightarrow$ restricționăm domeniul lui h la $\{2\}$
4. restricționarea nu afectează pe b, e, f

Intuitiv:

- inițial, după aplicarea restricțiilor unare, avem
  - $a \in \{3\}$
  - $d \in \{2\}$
  - $b, c, e, f \in \{1, 3\}$
  - $h \in \{1, 2\}$
  - $g \in \{4\}$
  - $i \in \{1\}$

4	2	1	a
b	c	d	4
e	f	g	2
h	4	3	i



1. a poate fi doar 3; a se învecinează cu d, i; ambele au domenii compatibile cu valoarea lui a
2. d poate fi doar 2, se învecinează cu b, c, g; sunt compatibile
3. i poate fi doar 1, se învecinează cu h, deci h nu poate fi 1 $\Rightarrow$ restricționăm domeniul lui h la $\{2\}$
4. restricționarea nu afectează pe b, e, f
5. rămân b, c, e, f cu domeniul $\{1, 3\}$

Pe cazul general, dacă o variabilă i poate avea doar valorile D'_i, ajustăm domeniile tuturor variabilelor vecine (care au restricții binare cu variabila i), în așa fel încât să conțină doar valori compatibile cu valorile din D'_i.

Dacă un domeniu D_k, al variabilei k vecină cu i, a fost ajustat, trebuie verificate din nou toate variabilele vecine cu variabila k, mai puțin variabila i.

Algoritmul AC-3 (*Arc Consistency 3*)

- trebuie ca toate restricțiile să fie unare sau binare
- putem forma un *graf* de restricții în care
  - nodurile corespund variabilelor
  - arcele corespund restricțiilor binare. Considerăm pentru fiecare restricție două arce orientate.
- un arc (X_i, X_j) corespunzător unei restricții R_{ij} este *arc-consistent* dacă

$$\forall x_i \in D_i, \exists x_j \in D_j, r_{ij}(x_i, x_j) = True$$

- dacă toate arcele din graf sunt arc-consistente, graful este *arc-consistent* și nu putem reduce mai mult (prin acest mecanism) spațiul de căutare

AC-3

1. reduce toate domeniile conform cu restricțiile unare
2. $Q \leftarrow \{(X_i, X_j) \mid R_{ij} \in R \text{ sau } R_{ji} \in R\}$
3. **cât timp** $Q \neq \emptyset$
4. $\quad (X_i, X_j) \leftarrow Q.pop()$
5. $\quad$ **dacă** $check(X_i, X_j)$ **atunci** *domeniul lui X_i s-a modificat*
6. $\quad\quad$ **dacă** $D_i = \emptyset$ **atunci întoarce** EȘEC
7. $\quad\quad Q = Q \cup \{(X_k, X_i) \mid R_{ki} \in R, k \neq j\}$
8. **întoarce** SUCCES

$check(X_i, X_j)$

1. **pentru** $x_i \in D_i$
2. $\quad$ **dacă** $\nexists x_j \in D_j \; . \; r_{ij}(x_i, x_j) = True$
3. $\quad\quad$ **atunci** $D_i \leftarrow D_i \setminus \{x_i\}$
4. **întoarce** *True* **dacă** D_i s-a modificat, **altfel** *False*

- complexitatea temporală pentru AC-3 este $O(e \cdot d^3)$, unde
  - $e = |R|$ – numărul de restricții (muchii)
  - $d = max_{i=\overline{1,N}}|D_i|$ – cardinalitatea maximă a domeniilor
- complexitatea spațială este $O(e)$
- algoritmi mai noi – dar mai complicați – pot fi mai buni. E.g., AC-4 are complexitate $O(e \cdot d^2)$

- dacă AC-3 întoarce EȘEC atunci înseamnă că problema nu are soluție
- dacă după realizarea AC-3, toate domeniile au cardinalitate 1 (spunem că *lățimea grafului de restricții* este 1), atunci nu mai este necesar backtracking
- dacă lățimea grafului de restricții este mai mare de 1, atunci
  - nu este garantat că problema are soluție, și
  - trebuie făcut backtracking pe variabilele cu domenii de cardinalitate > 1

Îmbunătățiri BKT

Reducerea spațiului de căutare prin detectarea timpurie a inconsistențelor:

- combinăm backtracking cu arc consistența, menținând arc-consistența pe măsură ce avansăm în recursivitate. (*Maintaining arc-consistency – MAC*)
- verificăm arc-consistența după ce atribuim o valoare în soluția parțială

BKT-MAC

1. **dacă** $Vars = \emptyset$ **atunci întoarce** Sol
2. $X_i \leftarrow alege_variabila(Vars)$
3. **pentru fiecare** $x \in D_i$
4. $\quad D_{copy} \leftarrow D$ — *copiem toate domeniile*
5. $\quad r \leftarrow AC\text{-}3'(arcs_out(X_i))$ — *arcs_out* întoarce toate arcele care pornesc din X_i; AC-3' este AC-3 începând de la pasul 3., cu Q pornind de la valoarea din argument
6. $\quad$ **dacă** r **și** $consistent(Sol \cup \langle X_i, x \rangle, R)$ **atunci**
7. $\quad\quad res \leftarrow BKT\text{-}MAC(Vars \setminus \{X_i\}, Sol \cup \langle X_i, x \rangle)$
8. $\quad\quad$ **dacă** $res \neq$ EȘEC **atunci întoarce** res
9. $\quad D \leftarrow D_{copy}$
10. **întoarce** EȘEC — *refacem domeniile*

Îmbunătățiri BKT : Ordonarea variabilelor

Putem ordona variabilele (funcția *alege_variabila*):

- aleator
- alegând variabila cea mai restricționată (cu domeniul cel mai redus) – *Minimum remaining value – MRV*
  - util atunci când ne dorim o singură soluție și vrem să forțăm, dacă e cazul, un eșec mai rapid
- alegând variabila cea mai puțin restricționată
  - util atunci când ne dorim aflarea tuturor soluțiilor și ne dorim un arbore de BKT mai lat și mai puțin adânc
- alegând variabila implicată în cele mai multe restricții cu variabile încă ne-atribuite – *Degree heuristic*

Putem parcurge valorile din domeniu într-o anumită ordine:

- selectăm întâi valoarea care elimină cele mai puține valori din variabilele vecine la menținerea arc-consistenței
  - util atunci când ne dorim o singură soluție și vrem să ajungem la o soluție mai repede
- selectăm întâi valoarea care elimină cele mai multe valori din variabilele vecine la menținerea arc-consistenței
  - util atunci când ne dorim toate soluțile și preferăm un graf mai puțin adânc

CSP Parțială

CSP Parțială

Pentru o problemă de tip CSP cu multe restricții, este posibil ca nu toate restricțiile să fie la fel de importante (vezi, de exemplu, problema orarului) $\Rightarrow$ CSP Parțială (*Flexible CSP*).

- asociem un cost cu fiecare restricție: $R_j = \langle t_j, r_j, c_j \rangle$,cu $c_j > 0$
- stabilim o limită suficientă S – considerăm o soluție Sol ca validă dacă $\sum_{j, r_j(Sol)=False} c_j \leq S$
- optimizare: dacă în parcurgere costul total al restricțiilor încălcate de atribuirea parțială curentă depășește S, sau depășește costul restricțiilor pentru cea mai bună soluție găsită până acum, nu are sens să mai continuăm pe aceast cale

PCSP – găsește prima soluție acceptabilă

Intrări: *Vars* – variabile rămase de atribuit, inițial X;
Sol – soluția (atribuirea) parțială, inițial $\emptyset$
cost – costul soluției parțiale *Sol*, cu S – limita suficientă

Ieșire: O atribuire completă de cost $\leq S$ sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci întoarce** *Sol*
2. $X_i \leftarrow$ *alege_variabila*(*Vars*)
3. **pentru fiecare** $x \in D_i$
4. $\quad Sol' \leftarrow Sol \cup \langle X_i, x \rangle$
5. $\quad c = cost + \sum_{X_i \in t_j, r_j(Sol')=False} c_j$
6. $\quad$ **dacă** $c \leq S$
7. $\quad\quad res \leftarrow PCSP(Vars \setminus \{X_i\}, Sol', c)$
8. $\quad\quad$ **dacă** $res \neq$ EȘEC **atunci întoarce** *res*
9. **întoarce** EȘEC

PCSP – găsește cea mai bună soluție acceptabilă

Intrări: *Vars* – variabile rămase de atribuit, inițial X;
Sol – soluția (atribuirea) parțială, inițial $\emptyset$
cost – costul soluției parțiale *Sol*, cu S – limita suficientă

Global: *sols* – toate soluțiile complete acceptabile găsite până acum
best_cost – costul minim al unei soluții din *sols*

Ieșire: O mulțime de atribuiri complete de cost $\leq S$, sau EȘEC

1. **dacă** $Vars = \emptyset$ **atunci**
2. $\quad sols \leftarrow sols \cup \{Sol\}$
3. $\quad$ **dacă** $cost < best_cost$ **atunci** $best_cost \leftarrow cost$
4. $X_i \leftarrow alege_variabila(Vars)$
5. **pentru fiecare** $x \in D_i$
6. $\quad Sol' \leftarrow Sol \cup \langle X_i, x \rangle$
7. $\quad c = cost + \sum_{X_i \in t_j, r_j(Sol')=False} c_j$
8. $\quad$ **dacă** $c \leq S$ **și** $c < best_cost$
9. $\quad\quad PCSP(Vars \setminus \{X_i\}, Sol', c)$ *va actualizza sols dacă este cazul*

Mulțumesc!



https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!